

Assets utilizados

AssetStore

3D 2D Add-Ons Audio AI Decentralization Essentials Templates Tools VFX **Sale** Sell Assets

You purchased this item on Jul 6, 2026.

Please rate and review this asset. Your honest review and rating will help other users who are deciding whether they should get this asset.

[Write a Review](#)



Four Evil Dragons Handpainted
Dungeon Mason

0:21 / 1:41

Dragon for Boss Monster : HP

Dungeon Mason ★★★★★ (26) | ♥ (2794)

FREE

👁 280 views in the past week

[Open in Unity](#) [♥](#)

B boa_net
★★★★★ 2 years ago
Free, cool, easy-to-use
Very nice. I saw it in a tutorial and used it as a basic enemy before I moved onto my own. Really helped me to keep going in my project, being able to...
[Read more reviews](#)

Los estudiantes deberán crear sus propios modelos 3D Maya/Blender durante el curso

¿Qué es Image Tracking?

Image Tracking (seguimiento de imagen) es una tecnología de Realidad Aumentada que permite a un dispositivo (celular, tablet, gafas AR) **reconocer una imagen específica impresa o en pantalla** y, al identificarla, **anclar contenido digital 3D** exactamente sobre esa imagen, como si el objeto virtual estuviera físicamente pegado a ella.

La analogía del "pegamento invisible"

Imagina que tomas una foto de tu rostro y se la muestras a un amigo. Tu amigo reconoce que eres tú porque conoce tus rasgos: la forma de tus ojos, tu nariz, la distancia entre ellos, la textura de tu piel. El **Image Tracking** hace exactamente eso, pero con una imagen impresa:

1. **Aprende** los rasgos únicos de la imagen (bordes, esquinas, texturas, contrastes).
2. **Guarda** esa "huella visual" en un archivo (como tu `.mind`).
3. **Busca** esa huella en lo que ve la cámara en tiempo real.
4. **Calcula** la posición y ángulo exactos de la imagen en el espacio 3D.
5. **Pega** el modelo 3D sobre esa imagen, ajustándose si mueves la cámara o la tarjeta.

¿Qué detecta exactamente?

No es magia. El algoritmo busca **features** (características visuales):

- **Esquinas y bordes** — donde cambia bruscamente el color.
- **Texturas complejas** — como el ruido Perlin de tu fondo.
- **Puntos de alto contraste** — esquinas de las alas del dragón, sus ojos, las puntas.

Por eso una hoja en blanco no sirve como marcador: no tiene features. Y por eso tu fondo procedural multicolor es tan efectivo: genera miles de esquinas y contrastes para que el algoritmo nunca se "pierda".

Image Tracking vs. otros tipos de AR

Tecnología	Qué necesita	Ejemplo
Image Tracking	Una imagen impresa específica	Tu tarjeta del dragón azul
Face Tracking	Un rostro humano	Filtros de Instagram
Plane Tracking	Una superficie plana cualquiera	IKEA Place (muebles en tu piso)
Marker Tracking	Un código QR o patrón blanco y negro	Marcadores de AR.js clásico
GPS AR	Tu ubicación geográfica	Pokémon GO

MindAR usa **Image Tracking avanzado**: no necesita un borde negro ni un patrón artificial. Funciona con cualquier imagen con suficiente detalle visual, como tu tarjeta del dragón.

En tu proyecto del libro pop-up

Esto es lo que sucede paso a paso cuando un alumno usa el libro:

1. Preparación (en el aula, por ti):

- Generas la imagen del dragón en Unity.
- La pasas por el compilador MindAR para crear `targets.mind`.
- Subes todo a GitHub Pages.

2. Impresión (en el libro):

- La tarjeta impresa contiene la misma imagen que "aprendió" el compilador.
- El QR apunta a la URL de GitHub.

3. Uso (por el alumno):

- Escanea el QR → abre la página web.
- Apunta la cámara a la tarjeta.
- El navegador compara lo que ve la cámara con la "huella" guardada en `targets.mind`.
- Si coinciden → ¡aparece el dragón!
- Si mueve el libro → el dragón se mueve con él, porque la cámara sigue rastreando la imagen.

¿Por qué se pierde a veces?

Como ya notaste, si desenfocas o alejas mucho, el dragón desaparece. Eso ocurre porque:

- La cámara ya no ve con claridad los **features** (se vuelven borrosos).
- La imagen ocupa muy pocos píxeles en pantalla (no hay suficiente detalle para comparar).
- Hay demasiada sombra o reflejo (cambia el contraste de los features).

Por eso en tu libro pop-up funciona tan bien: la tarjeta está **quieta, plana y bien iluminada** cuando la página está abierta. Es el escenario ideal para Image Tracking.

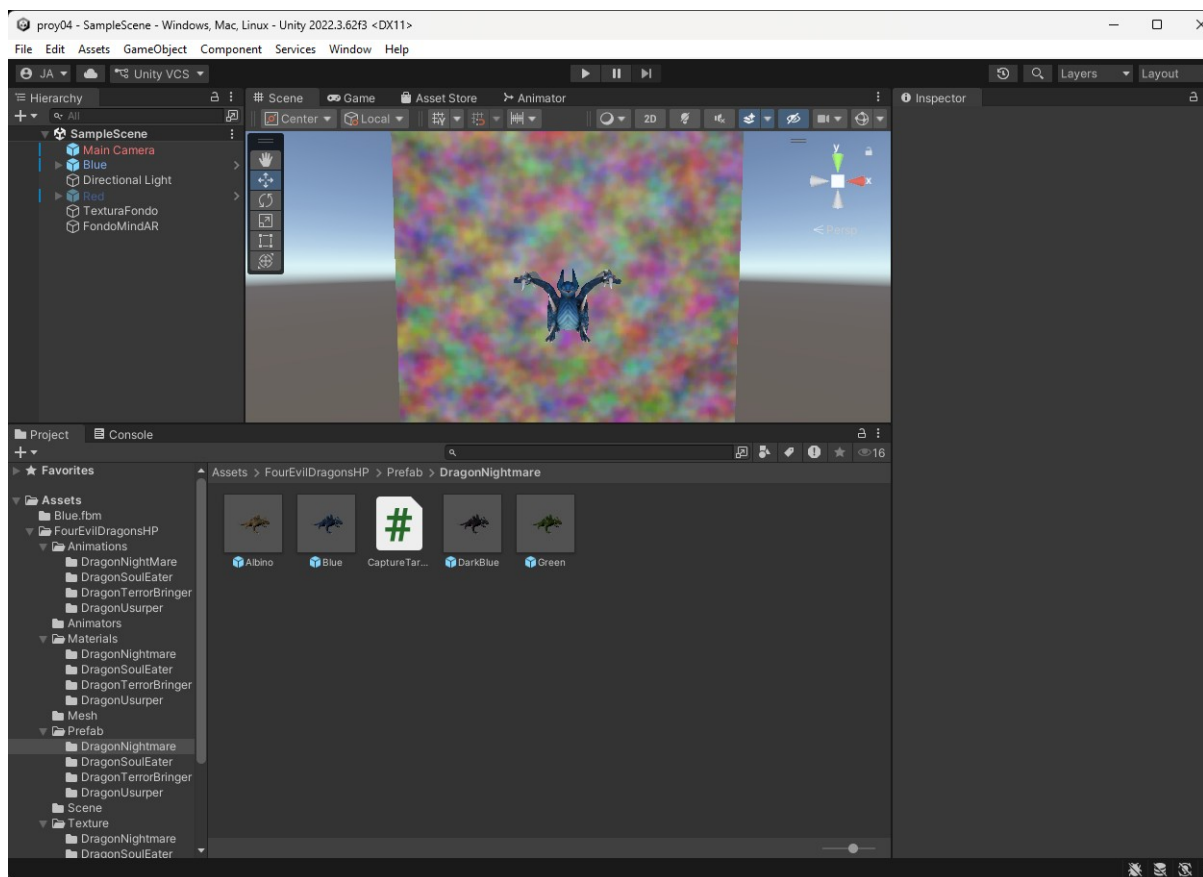
Guía: Crear Tarjetas MindAR desde Unity

1. Estructura de la Escena en Unity

Abre tu escena (SampleScene) y asegúrate de tener esta jerarquía:

SampleScene

- |— Main Camera
- |— Blue (dragón activo)
- |— Directional Light
- |— Red (dragón desactivado — para otra tarjeta)
- |— TexturaFondo (Empty con script)
- |— FondoMindAR (Cube con material)



2. Script para Generar la Textura de Fondo Procedural

Crea un script llamado `GenerarTexturaFondo.cs` y adjúntalo al objeto vacío `TexturaFondo`.

¿Por qué una textura procedural?

MindAR necesita **features visuales** (esquinas, bordes, texturas) para hacer tracking estable. Un fondo liso o de color sólido no funciona. Este script genera ruido Perlin fractal con alto contraste, creando miles de puntos de referencia automáticamente.

```
using UnityEngine;
using System.IO;
```

```
public class GenerarTexturaFondo : MonoBehaviour
{
    [Header("Configuracion")]
    public int width = 1024;
    public int height = 1024;
    public string nombreArchivo = "fondo_mindar.png";

    [ContextMenu("Generar Textura Fondo")]
    public void Generar()
    {
        Texture2D tex = new Texture2D(width, height, TextureFormat.RGB24, false);

        // Generar ruido Perlin con múltiples octavas (fractal)
        for (int x = 0; x < width; x++)
        {
            for (int y = 0; y < height; y++)
            {
                float nx = x * 0.01f;
                float ny = y * 0.01f;

                // Combinar múltiples frecuencias de ruido para crear caos
                float r = Mathf.PerlinNoise(nx, ny)
                    + Mathf.PerlinNoise(nx * 2.5f + 100, ny * 2.5f) * 0.5f
                    + Mathf.PerlinNoise(nx * 5f + 200, ny * 5f) * 0.25f;

                float g = Mathf.PerlinNoise(nx + 50, ny + 50)
                    + Mathf.PerlinNoise(nx * 3f + 150, ny * 3f) * 0.5f;

                float b = Mathf.PerlinNoise(nx + 100, ny + 100)
                    + Mathf.PerlinNoise(nx * 4f + 250, ny * 4f) * 0.5f;

                // Normalizar y aumentar contraste
                r = Mathf.Clamp01(r / 1.75f);
                g = Mathf.Clamp01(g / 1.75f);
                b = Mathf.Clamp01(b / 1.75f);
            }
        }

        tex.SetPixels32(tex.GetPixels32());
        tex.SaveAsImage(nombreArchivo);
    }
}
```

```

        // Aplicar contraste fuerte (equivalente a Niveles en GIMP)
        r = Mathf.Pow(r, 0.7f); // Gamma < 1 = más oscuro
        g = Mathf.Pow(g, 0.8f);
        b = Mathf.Pow(b, 0.9f);

        tex.SetPixel(x, y, new Color(r, g, b));
    }
}

tex.Apply();

// Guardar
byte[] bytes = tex.EncodeToPNG();
string folder = Path.Combine(Application.dataPath, "..", "TexturasGeneradas");
if (!Directory.Exists(folder)) Directory.CreateDirectory(folder);
string path = Path.Combine(folder, nombreArchivo);
File.WriteAllBytes(path, bytes);

Debug.Log($"<color=green>Textura generada:</color> {path}");

// Crear material automáticamente
Material mat = new Material(Shader.Find("Standard"));
mat.mainTexture = tex;
mat.SetFloat("_Smoothness", 0f);
mat.SetFloat("_Metallic", 0f);

// Guardar material como asset
#if UNITY_EDITOR
    UnityEditor.AssetDatabase.CreateAsset(mat, "Assets/MaterialFondoMindAR.mat");
    UnityEditor.AssetDatabase.SaveAssets();
#endif

    DestroyImmediate(tex);
}
}

```

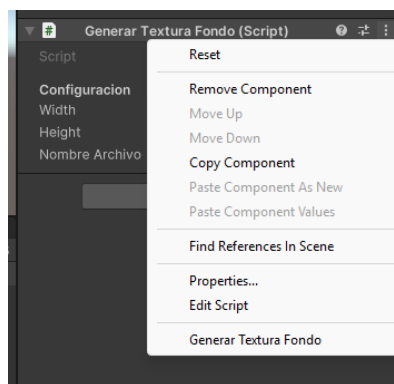
Explicación línea por línea:

Sección	Qué hace
<code>`width/height = 1024`</code>	Resolución cuadrada. MindAR funciona mejor con Potencias de 2.
<code>`Mathf.PerlinNoise`</code>	Genera ruido suave y orgánico.
<code>`nx * 2.5f + 100`</code>	Cada canal (R, G, B) usa **offsets diferentes ** para que el

<code>`/ 1.75f`</code>	resultado Sea multicolor, no gris. Normaliza la suma de amplitudes ($1 + 0.5 + 0.25 = 1.75$). Aumenta contraste empujando valores medios hacia blanco/negro. Más contraste = más features para	
<code>`Mathf.Pow(valor, 0.7f)`</code>	MindAR. Guarda fuera de `Assets` para no ensuciar el control de Versiones.	
<code>`TexturasGeneradas/`</code> <code>`Smoothness = 0, Metallic = 0`</code>	Material mate	evita reflejos que distorsionan la captura. la textura ya está guardada en el material
<code>`DestroyImmediate(tex)`</code>	Limpia memoria	Y en disco.

Cómo ejecutarlo:

1. Selecciona el objeto TexturaFondo en la jerarquía.
2. En el Inspector, haz **click derecho** sobre el componente del script.
3. Elige **Generar Textura Fondo**.
4. Unity creará fondo_mindar.png y el material MaterialFondoMindAR.mat.



3. El Fondo Físico: FondoMindAR

Crea un **Cube** en la jerarquía y configúralo así:

Propiedad Valor

****Position***

* X: 0, Y: 0, **Z: 2**

****Rotation***

* X: 0, Y: 0, Z: 0

****Scale**** **X: 4, Y: 4, Z: 0.1**

****Material*** `MaterialFondoMindAR` (o el que generaste)
*

Explicación

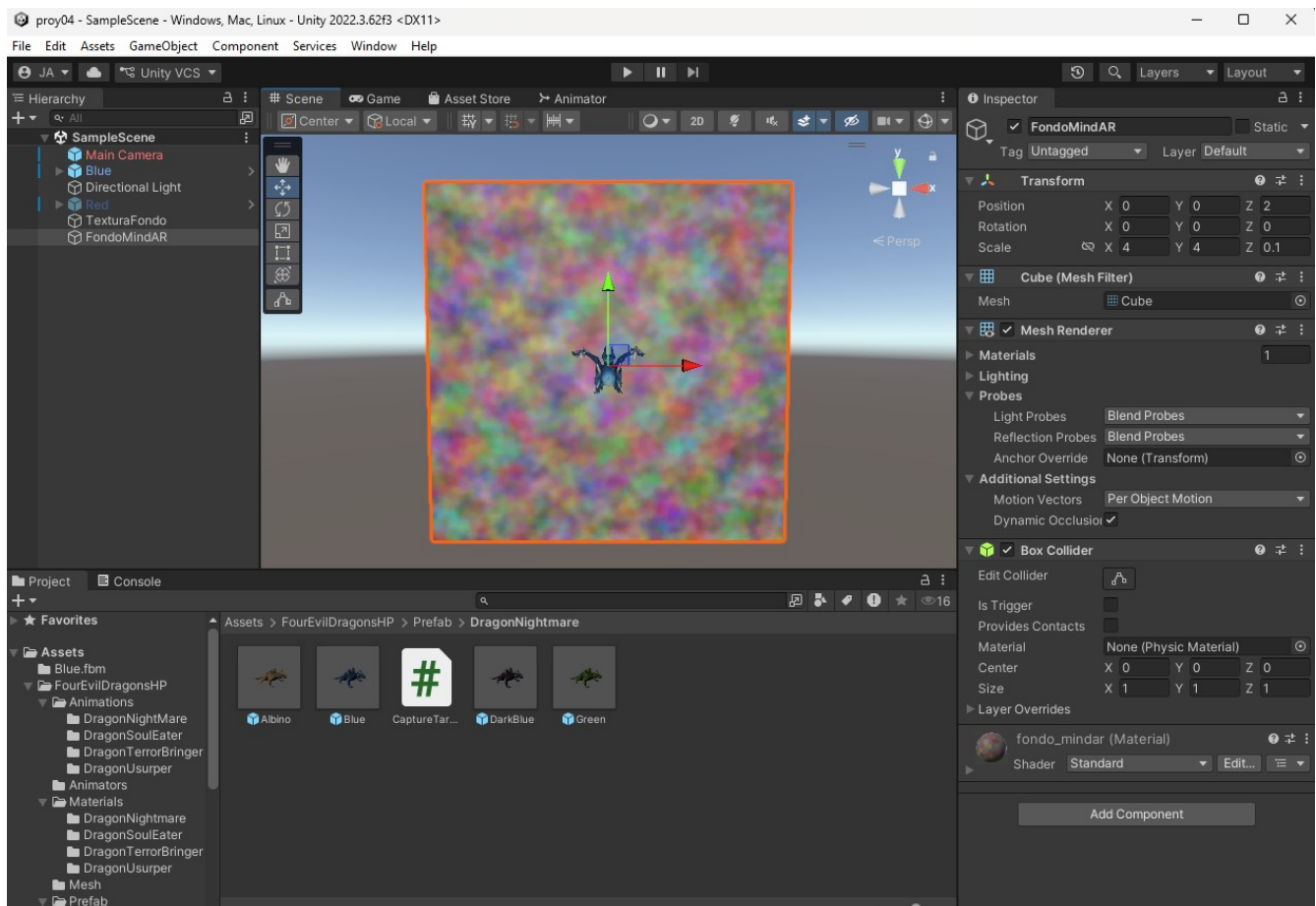
Detrás del dragón (que está en Z=0).

Plano perpendicular a la cámara.

Plano grande y delgado.

Textura procedural con ruido.

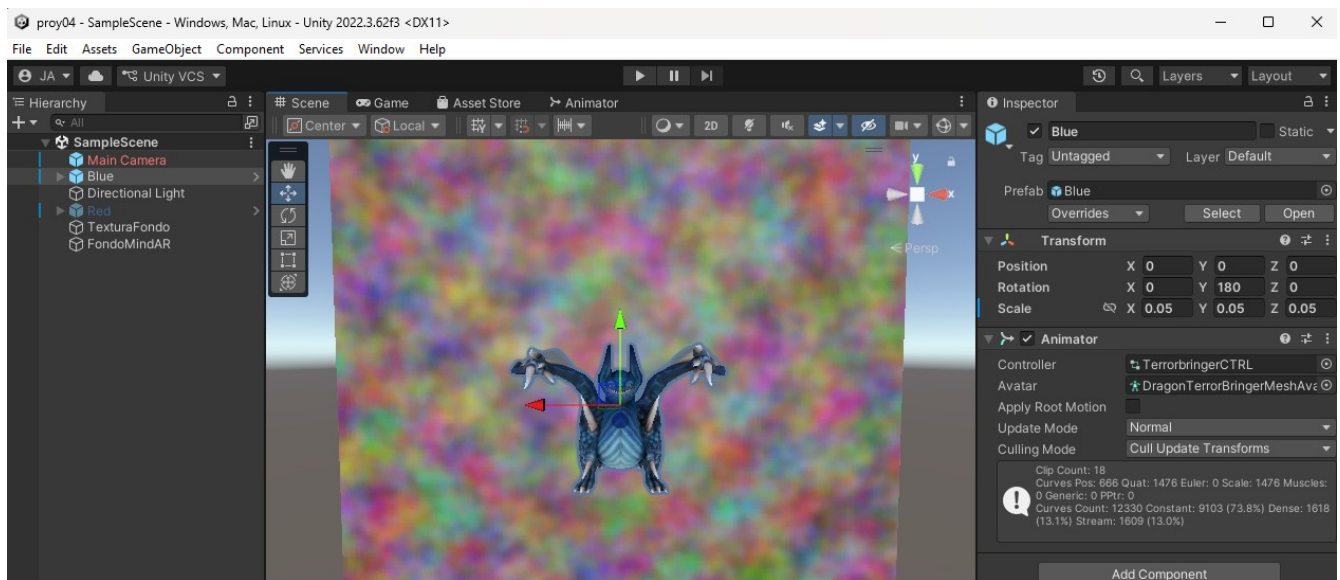
Nota: El Z: 0.1 en escala convierte el cubo en un plano fino. El Z: 2 en posición lo coloca detrás del dragón para que no ocluya.



4. Posicionamiento del Modelo (Blue)

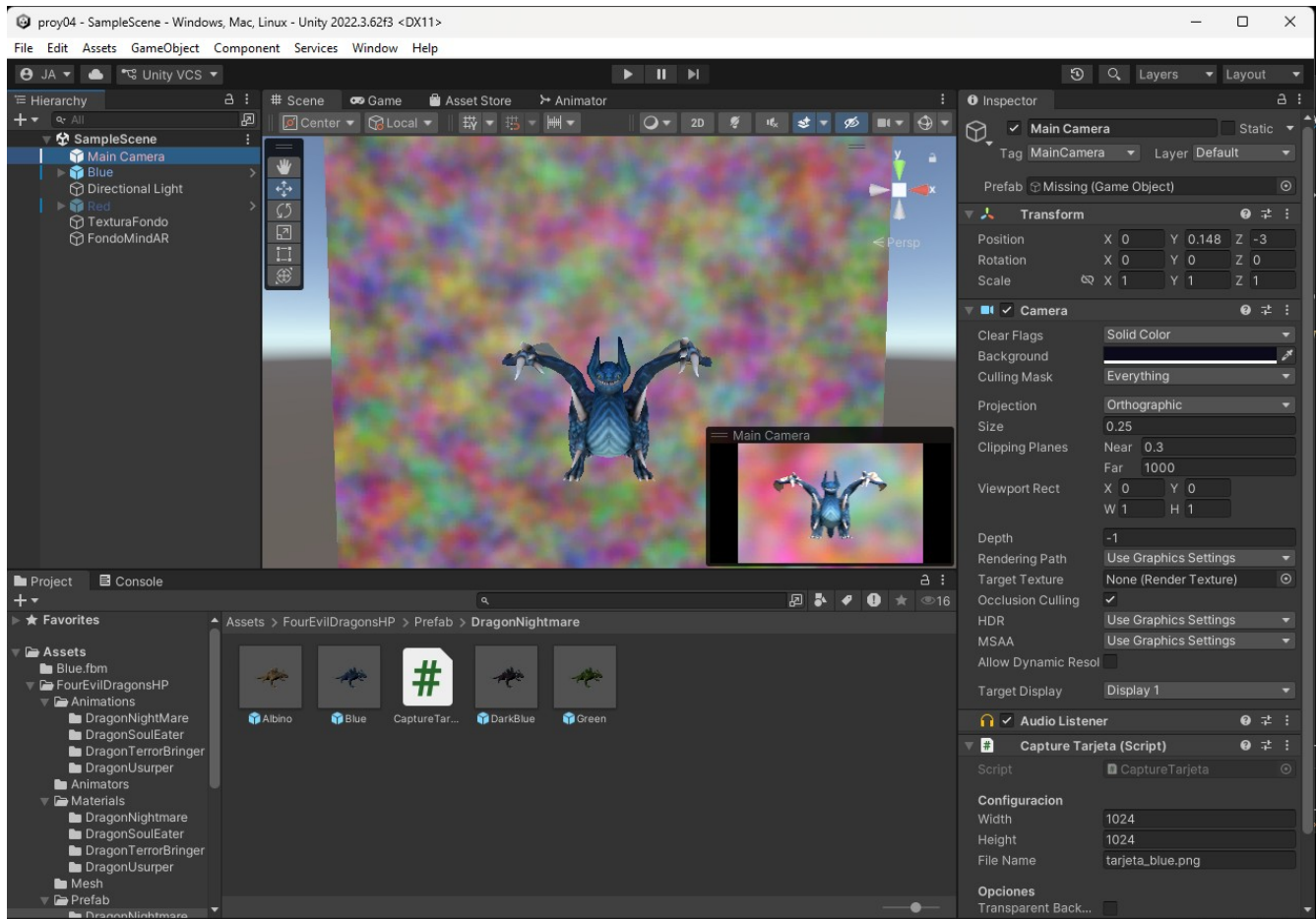
Arrastra tu prefab del dragón a la escena y configúralo:

Propiedad	Valor	Explicación	
**Position*		Centro del mundo, frente a la cámara.	
*	X: 0, Y: 0, Z: 0		
**Rotation*		El modelo original mira hacia atrás	180° lo orienta hacia la cámara.
*	X: 0, **Y: 180**, Z: 0		
	**X: 0.05, Y: 0.05, Z:		
Scale	0.05**	El modelo base es muy grande	se escala a tamaño tarjeta.



5. Configuración de la Main Camera

Selecciona Main Camera y configúrala para capturar la composición:



Propiedad	Valor recomendado
Position	Z: -5 (o ajusta hasta que el dragón llene el encuadre)
Rotation	X: 0, Y: 0, Z: 0
Projection	**Perspective**
Field of View	60
Background	Color sólido (ej. gris neutro) o Skybox
Clipping Planes	Near: 0.3, Far: 1000

Importante: La cámara debe capturar tanto el dragón como el fondo completo. Ajusta la posición Z de la cámara hasta que el dragón ocupe el centro y el fondo llene los bordes.

Script de la Cámara

```
using UnityEngine;  
using System.IO;
```

```
#if UNITY_EDITOR  
using UnityEditor;
```

```
#endif
```

```
public class CaptureTarjeta : MonoBehaviour  
{
```

```
    [Header("Configuracion")]
```

```
    public int width = 1024;
```

```
    public int height = 1024;
```

```
    public string fileName = "tarjeta.png";
```

```
    [Header("Opciones")]
```

```
    public bool transparentBackground = false;
```

```
    public Color backgroundColor = new Color(0.04f, 0.04f, 0.10f, 1f);
```

```
    [ContextMenu("Capturar Tarjeta")]
```

```
    public void Capturar()  
{
```

```
        Camera cam = GetComponent<Camera>();
```

```
        // Guardar estado original
```

```
        CameraClearFlags flagsOriginales = cam.clearFlags;
```

```
        Color colorOriginal = cam.backgroundColor;
```

```
        // Configurar camara
```

```
        cam.clearFlags = CameraClearFlags.SolidColor;
```

```
        cam.backgroundColor = transparentBackground
```

```
            ? new Color(0, 0, 0, 0)
```

```
            : backgroundColor;
```

```
        // Crear RenderTexture con anti-aliasing
```

```
        RenderTexture rt = new RenderTexture(width, height, 24,
```

```
        RenderTextureFormat.ARGB32);
```

```
        rt.antiAliasing = 4;
```

```
        rt.Create();
```

```
        cam.targetTexture = rt;
```

```
        RenderTexture.active = rt;
```

```
        // Renderizar
```

```
        cam.Render();
```

```
        // Leer pixeles
```

```
        TextureFormat formato = transparentBackground ? TextureFormat.RGBA32 :
```

```
        TextureFormat.RGB24;
```

```
        Texture2D screenShot = new Texture2D(width, height, formato, false);
```

```
        screenShot.ReadPixels(new Rect(0, 0, width, height), 0, 0);
```

```
        screenShot.Apply();
```

```
        // Guardar PNG en bytes (AQUI VA, antes de usarlo)
```

```
        byte[] bytes = screenShot.EncodeToPNG();
```

```
        // Restaurar camara
```

```
        cam.targetTexture = null;
```

```
        RenderTexture.active = null;
```

```
        cam.clearFlags = flagsOriginales;
```

```
        cam.backgroundColor = colorOriginal;
```

```
        // Liberar RenderTexture
```

```
        rt.Release();
```

```
        DestroyImmediate(rt);
```

```
        // Liberar Texture2D
```

```

DestroyImmediate(screenShot);

// Guardar PNG DIRECTAMENTE EN ASSETS (sin subcarpeta)
string folder = Application.dataPath;

// Numeracion automatica para no sobrescribir
string path = Path.Combine(folder, fileName);
if (File.Exists(path))
{
    int i = 1;
    string nombreBase = Path.GetFileNameWithoutExtension(fileName);
    string extension = Path.GetExtension(fileName);
    while (File.Exists(Path.Combine(folder, $"{nombreBase}_{i:D3}{extension}")))
    {
        i++;
    }
    path = Path.Combine(folder, $"{nombreBase}_{i:D3}{extension}");
}

File.WriteAllBytes(path, bytes);

// REFRESCAR ASSETS PARA QUE UNITY LO MUESTRE
#if UNITY_EDITOR
AssetDatabase.Refresh();
#endif

Debug.Log($"<color=green>Guardado en:</color> {path}");
}
}

```

6. Iluminación

Usa la misma configuración que en tu escena:

1. **Directional Light** (ya viene por defecto):

- Rotación aproximada: X: 50, Y: -30, Z: 0.
- Intensidad: 1 (o ajusta para que el dragón se vea con volumen).

2. **Ambient Light** (Window → Rendering → Lighting → Environment):

- Source: Color o Skybox.
- Intensidad: 1.

Esto da sombras suaves y volumen al modelo, haciendo que la tarjeta se vea profesional.

7. Capturar la Imagen Final (PNG)

Captura con el script de Cámara

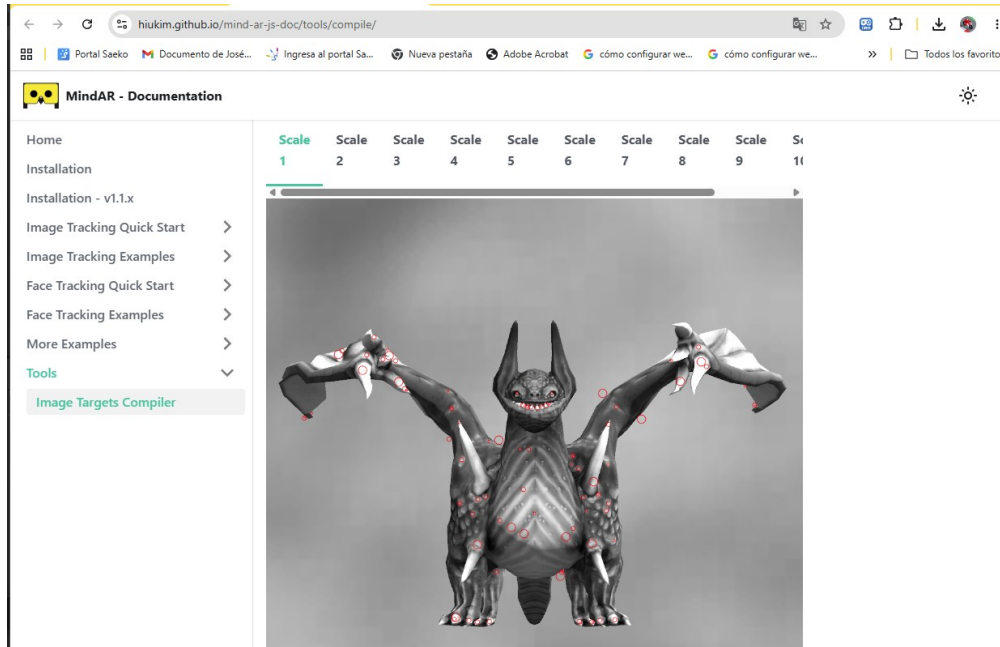
8. Verificar la Imagen en el Compilador MindAR

Abre el compilador oficial:

<https://hiukim.github.io/mind-ar-js-doc/tools/compile/>

Pasos:

1. Arrastra tu `tarjeta_blue.png`.
2. Haz clic en **Start**.
3. Observa los **puntos rojos** (feature points) sobre la imagen.



¿Qué buscar?

- ☒ Puntos distribuidos por **toda la imagen** (fondo + dragón).
- ☒ Densidad alta en bordes del dragón y alas.
- ☒ No hay zonas grandes sin puntos.

Si la distribución se ve bien, haz clic en **Download** y obtendrás el archivo `targets.mind`.

9. Integrar en tu Proyecto WebAR

Coloca los archivos en tu estructura de GitHub:

Lab06/

```
├── index.html
├── targets.mind    ← El archivo descargado del compilador
├── models/
│   └── Blue.glb
```

